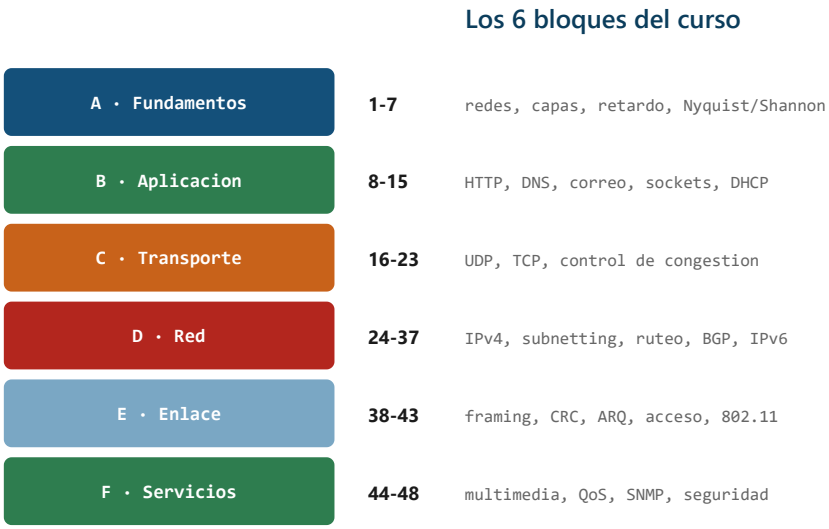
Clase 37 — ICMP, ping y traceroute · Repaso de red y Simulacro 4



Bloque: D (Capa de Red) · **Semana:** 13 · **Duración estimada:** 180 min (≈70 min teoría + 110 min repaso y simulacro) **Correlativa:** Clase 36 (IPv6) · **Sigue con:** Clase 38 (Capa de enlace: servicios y framing) **Material de referencia:** Tanenbaum cap. 5 (§5.4.1) · RFC 792, RFC 1122, RFC 4443, RFC 8201

🎯 Al terminar esta clase vas a poder

1. Explicar **por qué existe ICMP** (el problema de anidamiento), su formato, y por qué **no es un protocolo confiable** ni genera errores sobre errores.
2. **Reconstruir de memoria la tabla de tipos y códigos de ICMP**, incluidos los **obsoletos** (Source Quench, Address Mask).
3. Explicar qué hace `ping`, qué mide, y por qué es **la herramienta de diagnóstico del PMTU**.
4. Explicar **cómo funciona** `traceroute`: el campo TTL, los dos sabores (UDP e ICMP), los **tipos de mensaje** involucrados y **los nodos que colaboran**.
5. Explicar **las limitaciones honestas** de `traceroute`: el `*` no significa caída, y el camino de vuelta puede ser distinto.
6. **Repasar en forma densa** las clases 24 a 37: fórmulas, reglas, números de ICMP, atributos de BGP, mensajes de OSPF, modelos de IPv6.
7. **Rendir un simulacro completo** de 100 puntos en 180 minutos.

> **Prerequisito del examen:** `2011-03 P3` (6 pts — **principio de funcionamiento de** `traceroute`, tipos de mensaje y nodos que colaboran), `2010-12 Problema 2` (la **tabla de tipos y códigos de ICMP** es material de estudio por sí sola), `2010-12 P5` (IPv6), más todo el bloque D.

PARTE A — TEORÍA: ICMP, PING Y TRACEROUTE

A.1 Por qué existe ICMP

El problema de fondo: los errores de IP no se pueden reportar en IP.

```
Un paquete IP llega a un router y no se puede entregar.
¿Cómo se lo avisas al emisor?

Opción A: mandar OTRO paquete IP de error.
⇐ Pero si el problema es que el destino está inalcanzable, o que la red
está caída, o que hay un loop, el paquete de ERROR tampoco va a llegar.
⇐ Y si lo que falló fue el PROPIO router (TTL vencido, tabla de ruteo
vacía), el paquete de error tiene que salir por una ruta que
probablemente tampoco funciona.

⇐ IP no puede hablar de sí mismo: es un problema de ANIDAMIENTO.
```

La solución: un protocolo de capa 3 paralelo, definido en `RFC 792` (1981), que viaja **dentro** de IP pero es conceptualmente distinto: los errores van **de un nodo a otro**, no de un host a un host de capa superior.

ICMP = *Internet Control Message Protocol*. Sus dos funciones:

1. **Diagnóstico y control de errores**: reportar que un paquete no pudo entregarse.
2. **Diagnóstico de red**: `ping`, `traceroute`.

A.2 El formato del mensaje ICMPv4

```
┌──────────┬──────────┬────────────────────┬──────────────────────────────┐
│ Type     │ Code     │ Checksum           │ Cuerpo…                      │
│ (tipo)   │(código)  │                    │ (varía según tipo y código)  │
│ 8 bits   │ 8 bits   │ 16 bits            │                              │
└──────────┴──────────┴────────────────────┴──────────────────────────────┘
byte 0 byte 1 bytes 2–3

Ejemplo: un mensaje de "Time Exceeded" (tipo 11) lleva en el cuerpo:
los 8 bytes del datagrama que expiró (cabecera IP) + los primeros 64
bytes de ese datagrama.
⇐ ¿Por qué? Para que el emisor pueda IDENTIFICAR el paquete que
expiró sin tener que esperar a que llegue a la capa 4.
```

Para `ping` y `traceroute` los dos campos que importan son los del cuerpo:

Campo	Tamaño	En Echo Request/Reply
Identifier	16 bits	Identifica **qué par de `ping`** es este (número de proceso o secuencia)
Number / Sequence	16 bits	El **número de secuencia** dentro de ese `ping`. Permite emparejar request con reply y detectar pérdida

El campo `Code`, de memoria: hoy casi solo se usa `0`, pero en **Destination Unreachable** es donde tiene todo el sentido (ver la tabla de §A.3).

A.3 La tabla de tipos y códigos de ICMP — de memoria

> ⚠️ **El `2010-12 Problema 2` incluye la tabla oficial de tipos y códigos como dato del examen. Esa tabla es material de estudio por sí sola.** Reconstruila entera y sabé en particular **los códigos del tipo 3**.

Los tipos (0 a 12), con lo que hay que saber de cada uno:

Tipo	Nombre	Qué es / para qué
0	**Echo Reply**	Respuesta de `ping`. **No es un error.**
3	**Destination Unreachable**	El paquete no pudo entregarse. **Sus códigos importan mucho** (tabla de abajo).
4	**Source Quench**	**OBSOLETO.** "Estás mandando demasiado rápido, frená". **Hoy no lo genera nadie.** Ver abajo.
5	**Redirect**	"Hay un camino mejor". **Casi no se usa en IPv4 moderno** (ver abajo).
8	**Echo Request**	El `ping`. **No es un error.**
9	**Router Advertisement**	Un router se anuncia (usado por RIP y por *router discovery*)
10	**Router Solicitation**	Un host pide que los routers se anuncien
11	**Time Exceeded**	**TTL se venció en tránsito.** El que usa `traceroute`
12	**Parameter Problem**	Un campo de la cabecera IP es inválido
13	**Timestamp / Timestamp Reply**	reloj de un nodo. **Obsoleto en la práctica**
17	**Address Mask Request / Reply**	**OBSOLETO.** Pide la máscara de subred. **Reemplazado por DHCP**

Los CÓDIGOS del tipo 3 (Destination Unreachable) — la parte que más se pregunta, tal como figura en la tabla del `2010-12`:

```
0 Net Unreachable ← la RED no existe
1 Host Unreachable ← el HOST no existe
2 Protocol Unreachable ← no hay proceso para ese protocolo
3 Port Unreachable ← no hay proceso escuchando en ese puerto
4 Fragmentation Needed and Don't Fragment was Set ← el que dispara el PMTUD
5 Source Route Failed ← falló una ruta source-routed
6 Destination Network Unknown
7 Destination Host Unknown
8 Source Host Isolated ← obsoleto
9 Communication with Destination Network is Administratively Prohibited
10 Communication with Destination Host is Administratively Prohibited
12 Parameter Problem ← también existe como tipo 12
```

⚠️ **Sobre la lectura de esa tabla en el PDF del 2010-12:** el examen la imprime en **dos columnas desalineadas** (la de nombres de tipos y la de códigos del tipo 3 quedaron desfasadas una posición), así que el texto extraído parece corrido. Leída correctamente es la tabla canónica de `RFC 792`, que es la de arriba. **Lo que hay que retener sin ambigüedad:** el **3/4** ("Fragmentation Needed and DF Set", el del PMTUD) y el **3/3** ("Port Unreachable", el que cierra `traceroute` en su sabor UDP).

Los que están OBSOLETOS, explícitamente:

- `Source Quench` **(tipo 4) — OBSOLETO.** La idea era que el router le dijera al emisor "estás saturando, mandá menos". **Hoy no lo genera nadie**, por una razón concreta: **creaba una tormenta de mensajes.** Un router congestionado generaba un Source Quench por cada paquete descartado, lo que **aumentaba la congestión**. La.stderr se resolvió de otra forma: **dejar que la pérdida sea la señal** (que es lo que hacen los algoritmos de control de congestión) y, mejor aún, con **ECN**, que marca en lugar de descartar. ⚠️ **No lo propongas nunca como solución a un problema de congestión.**
- `Address Mask Request/Reply` **(tipo 17) — OBSOLETO.** Servía para que un host **descubriera la máscara de subred** por broadcast. **Hoy la máscara viene en el DHCP**, y en IPv6 la dirección ya trae el prefijo. Es historia pura.

`Redirect` **(tipo 5) — por qué casi no se usa en IPv4 moderno:**

```
El Redirect dice "hay un camino mejor que este, andá por allá". El problema:
para que sea confiable, el host tendría que CONFIRMAR que el router que se lo
dice es legítimo. Con ARP sin autenticar, un atacante puede inyectar Redirects
y hacer traffic blackholing.

Además, en IPv4 los routers el Redirect solo entreStub (routers de un
único enlace), por lo que casi nunca se usa entre routers de tránsito.

⇐ En IPv6 el equivalente (tipo 137) existe y es más usado, porque ND sí
tiene mecanismos de seguridad asociados (Secure NDP / SEND).
```

A.4 ICMP no es confiable, y no genera errores sobre errores

Dos reglas de diseño que hay que saber de memoria:

1. ICMP no es un protocolo de entrega confiable. No hay ACK de los ACKs, no hay retransmisión, no hay control de congestión. Un ICMP **se puede perder** como cualquier otro paquete, y **no se retransmite**.

```
⇐ Consecuencia directa: un "Destination Unreachable" perdido NO genera
otro error. El emisor simplemente no se entera de que su paquete falló.
⇐ Y esto es lo que hace que los Firewalls puedan filtrar ICMP "sin romper
la aplicación"... salvo que se need el PMTU (black hole, Clase 35).
```

2. No se generan mensajes ICMP sobre mensajes ICMP (con excepciones estrictas):

```
⇐ Por qué: si un router no puede entregar un ICMP, no emite otro ICMP
diciendo que no pudo entregar el ICMP. Y así, hacia atrás, indefinidamente.
Sería una tormenta garantizada.

⇐ Las EXCEPCIONES (que hay que conocer):
- ICMP tipo 3 código 4 (Fragmentation Needed): SÍ puede sobre otro
ICMP, porque el router necesita que el emisor reduzca el tamaño.
- ICMP tipo 3 código 2/3 sobre mensajes ICMP: en la práctica los routers
modernos SÍ los generan (por ejemplo, un "port unreachable" porque nadie
escucha ICMP en ese puerto).
⇐ La regla de "no generar sobre ICMP" es de la era de RFC 1122 y los
routers reales la relajaron en estos casos concretos.
```

El detalle fino del Report Time Exceeded con datos del datagrama que expiró:

```
ICMP Time Exceeded lleva en el cuerpo:
┌──────────────────────────────────┬──────────────────────────────┐
│ cabecera IP del datagrama que │ primeros 64 bytes del │
│ expiró (20 bytes) │ datagrama original │
└──────────────────────────────────┴──────────────────────────────┘

¿Por qué? Porque el emisor puede así RECONOCER qué paquete se perdió y
(en el caso de traceroute) Extract los datos que le interesan.

En el caso de un Echo Request de traceroute, esos 64 bytes incluyen el
campo Identifier y el Number Sequence ⇒ el emisor puede emparejar el
Time Exceeded con el probe que lo generó.
```

⚠️ Actualización a 2026 — ICMP no envejeció, pero su entorno sí

Sigue igual — de memoria, sin dudar: - El **formato** del mensaje ICMPv4: tipo, código, checksum de 16 bits, identificador, número de secuencia. **No cambió en 30 años.** - Los **números** de tipo y código que se piden: 0, 3 (con sus 13 códigos), 8, 11, 12, 13, 17. - El principio de **traceroute**: manipular el **TTL** para que cada router intermedio devuelva un error y así revelar el camino de ida. - Que **un router nunca genera ICMP sobre otro ICMP**, y que ICMP **no es confiable** (va sobre IP, no sobre TCP).

Cambió o seerteó: - `Source Quench` **(tipo 4) y** `Address Mask` **(tipo 17) están obsoletos.** El primero se usaba para pedir que el emisor bajara la tasa; se demostró que era **contraproducente** (el emisor respondsba al cuelgue de la app, no a la red) y hoy **nadie lo genera**. El segundo quedó inútil con CIDR. Si un apunte los lista como vigentes, está viejo. - `ICMPv6` **(** `RFC 4443` **) es obligatorio en IPv6** y es mucho más rico que ICMPv4: incluye **Neighbor Discovery** (RS, RA, NS, NA, Redirect), **Packet Too Big** (tipo 2), **Parameter Problem** (tipo 4) y **Path MTU Discovery** (`RFC 8201`). Cuando pregunten "ICMP" en un contexto de IPv6, la respuesta es **ICMPv6**, no ICMPv4. - **Los mensajes de control de** `traceroute` **hoy se filtran más.** Por eso `mtr` (que combina `ping` y `traceroute` con losses por nodo) es más útil que `traceroute` a secas, y por eso existen las **capturas pasivas** de flows. - `ping -M do -s <tamaño>` **sigue siendo la herramienta de diagnóstico del PMTU** en Linux, y `tracepath` muestra la MTU de cada salto directamente. Es la respuesta moderna al **PMTUD black hole** cuando el ICMP no vuelve.

Consecuencia para el examen: la Pregunta 3 de 2011-03 (traceroute) y la Pregunta 5 de 2010-12 (tipos y códigos de ICMP) se responden **exactamente como en 2012**. Lo único que puede cambiar es que te pidan marcar los tipos obsoletos.

A.5 `ping`

Qué es, mecánicamente:

```
⇐ Emisor ──────── ICMP Echo Request (tipo 8, código 0) ──────────► Receptor
◄──────── ICMP Echo Reply (tipo 0, código 0) ──────────────────────

⇐ En la cabecera del Echo Request va un campo IDENTIFIER (id de proceso)
y un campo SEQUENCE NUMBER (número de secuencia).
⇐ El Reply copia esos mismos valores y calcula el RTT.
```

Qué mide, en la práctica:

```
RTT = (t_recv - t_send)

Repetir N veces (por defecto 4 en Unix) y sacar:
- el RTT mínimo, el promedio, el máximo
- la desviación estándar (jitter)
- el PORCENTAJE DE PÉRDIDA

⇐ El INTERESANTE no es el RTT absoluto sino la VARIACIÓN.
Una línea base de 40 ms es una red sana.
Una línea base de 40 ms que a veces sube a 300 ms tiene cola (congestión).
```

El `ping` grande con DF como herramienta de diagnóstico del PMTU — esto es lo que hay que acordarse:

```
# Linux: -M do = poner DF=1 ; -s = tamaño del payload ICMP
ping -M do -s 1400 ejemplo.com # si falla → el cuello de botella está
 # por debajo de 1400 + headers
ping -M do -s 1000 ejemplo.com # si funciona → el cuello está entre
 # 1000 y 1400
ping -M do -s 1400 -c 3 ejemplo.com # 3 intentos, para distinguir de ruido

# tracepath muestra el PMTU de cada salto directamente
tracepath ejemplo.com

# Windows: ping -f -l 1400 ejemplo.com (-f = DF, -l = tamaño del payload)
```

⚠️ **El `ping` CON DF tiene un detalle importante:** si el payload es demasiado grande y el firewall **filtra** el ICMP tipo 3 código 4, el `ping` muestra `Request timeout` en vez de `Frag needed and DF set`. **Eso es el black hole** (Clase 35, §1.9). Un `ping` normal (sin DF) fragmenta y por eso "funciona", mientras que las conexiones reales (que usan DF) no. **Esa asimetría es el síntoma clásico.**

A.6 `traceroute` — el Examen 2011-03 P3

Este es el apartado central de la Parte A y vale **6 puntos**. Vamos por partes.

La idea

`traceroute` **exploita el campo TTL** de la cabecera IP. El TTL es un contador que **cada router decrementa en 1**. Cuando el TTL llega a **0**, el router **descarta el paquete** y le manda al emisor un **ICMP "Time Exceeded"** (tipo 11, código 0, "TTL expired in transit").

```
⇐ El truco es: mandar un paquete con TTL = 1, después TTL = 2, después 3...

⇐ El router que está a N saltos recibe el paquete con TTL = N.
Lo decrementa a 0. Lo descarta. Y devuelve un Time Exceeded.
⇐ Ese mensaje Time Exceeded **TRAE el nombre/IP del router**.
⇐ ⇐ ⇐ ESO es el mecanismo: cada router "delata" su existencia al
decir "yo recibí tu paquete y lo tiré".

⇐ Así, con N paquetes, obtenemos el camino COMPLETO de N saltos.
```

```
⇐ Ejemplo simplificado (supongamos 4 saltos hasta el destino):

Probe 1: TTL=1
H1 ──────────▶ [R1] TTL 1→0, descarta, manda Time Exceeded
◀────────────── (de R1)

Probe 2: TTL=2
H1 ─────────────────▶ [R2] TTL 2→1 (pasa); [R1] pasa; [R2] 1→0,
descarta, manda Time Exceeded
◀──────────────────────────────────── (de R2)

Probe 3: TTL=3
H1 ──────────────────────────▶ [R3] TTL 3→0, manda Time Exceeded
◀──────────────────────────── (de R3)

Probe 4: TTL=4
H1 ──────────────────────────────────▶ [R4 = DESTINO]
◀──────────────────────────── (respuesta del destino, NO Time Exceeded)
⇐ Cuando la respuesta NO es "Time Exceeded", llegamos al destino.
```

Los dos sabores

	Sabor UDP	Sabor ICMP
Por defecto en	Unix, Linux, BSD, **macOS**	**Windows**
Qué manda	Paquetes **UDP** a un **puerto destino alto** (por defecto **~33434**), con probabilidad de uso muy baja	Paquetes **ICMP Echo Request** (tipo 8, código 0)
Por qué puerto alto	Para Maximizar la probabilidad de que **nadie esté escuchando** en ese puerto ⇒ generar el error esperado	—
Qué llega del destino	**ICMP "Destination Unreachable" / "Port Unreachable"** (tipo **3**, código **3**), porque nadie escucha en ese puerto	**ICMP Echo Reply** (tipo **0**, código 0)
En resumen	`(puerto destino = 33434) + (respuesta = tipo 3 código 3)`	`(tipo 8) + (respuesta = tipo 0 código 0)`

⚠️ **El traceroute basado en ICMP, al llegar al destino, es básicamente `ping`.**

El detalle fino: ¿por qué el sabor UDP es "mejor" para diagnóstico, aunque su configuración suene rara?

```
⇐ El sabor UDP usa un puerto ALTO (33434) que casi seguro no está en uso.
⇐ Cuando llega al destino, el host receptor ve un paquete UDP a un
puerto donde no hay nadie escuchando → responde "Port Unreachable".
⇐ Esa respuesta la genera el KERNEL del host final, sin ninguna
aplicación que se entere. Es muy reliable.

⇐ En el sabor ICMP, cuando llega al destino, ÉSTE responde con Echo Reply.
⇐ Eso significa que el host destino tiene que GENERAR respuestas ICMP.
⇐ Muchos sistemas (servidores Windows, por ejemplo) tienen el ICMP
Echo RESPUESTO DESHABILITADO por firewall.
⇐ Por eso en Windows, traceroute a veces muestra "*" en el destino.
```

Los mensajes involucrados (los números exactos)

```
⇐ Cada router que recibe un paquete con TTL = 1 (o el valor que toque):
1. lo DECREMENTA a 0
2. lo DESCARTA
3. manda un ICMP "Time Exceeded" — TIPO 11, CÓDIGO 0
(= "TTL expired in transit")

⇐ Por defecto 3 paquetes por instancia (3 probes por TTL):
⇐ Para no perder información por una sola pérdida de ICMP.
⇐ Los 3 resultados se muestran como 3 RTTs en la línea.

⇐ Cuando el TTL es suficientemente alto para llegar al destino:
⇐ EN EL SABOR UDP: el destino responde con ICMP
"Destination Unreachable" / "Port Unreachable" — TIPO 3, CÓDIGO 3
(porque nadie escucha en ese puerto).
⇐ EN EL SABOR ICMP: el destino responde con Echo Reply — TIPO 0, CÓDIGO 0.

⇐ El límite por defecto es 30 saltos (suficiente para Internet).
```

Los nodos que colaboran, y el punto clave:

```
⇐ 1. El EMISOR: genera los probes con TTL incremental, mide los RTTs, y
muestra las respuestas.
⇐ 2. CADA ROUTER INTERMEDIO: decrementa el TTL, y cuando llega a 0, lo
descarta y genera un Time Exceeded.
⇐ 3. El DESTINO: en el sabor UDP, genera un Port Unreachable; en el sabor
ICMP, un Echo Reply.

⇐ ⚠️ PUNTO CLAVE: los nodos que responden pueden estar en un CAMINO
DISTINTO al de ida, porque el camino de vuelta no tiene que ser el
mismo. Los tiempos que muestra traceroute son RTTs de IDA Y VUELTA,
y esa vuelta puede tomar otra ruta.

⇐ ⚠️ Y si un nodo filtra ICMP (o su respuesta se pierde), aparece como "*".
Eso NO significa que esté caído.
```

Las limitaciones honestas

Son tres, y la tercera es la que más se pregunta:

(1) El `*` NO significa que el router esté caído.

```
⇐ El "*" significa: NO RECIBÍ NADA.
⇐ Hay (al menos) cuatro razones por las que no recibís nada, sin que el
router esté caído:
a) El router FILTRA el ICMP de tipo 11 por política de seguridad
(es muy común en empresas y en carriers).
b) El ICMP de VUELTA no llega (filtro en el camino de regreso).
c) El PROBE se perdió en el camino (pérdida normal en Internet).
d) El router es un punto de salto que no reenvía ICMP de origen externo.

⇐ Por eso el "*" es AUSENCIA DE EVIDENCIA, no evidencia de caída.
⇐ El "*" SÍ es informativo cuando es el ÚLTIMO salto (cuando además los
siguientes tampoco responden) — eso sí sugiere que el destino o su
firewall están bloqueando ICMP, o que el destino está caído.
```

(2) El camino de vuelta puede ser distinto del de ida.

```
⇐ En Internet, el camino de ida y el de vuelta de una comunicación
NO tienen por qué ser el mismo (ruteo asimétrico).

⇐ Consecuencia: el "RTT" que muestra traceroute es el RTT del ciclo
completo (ida + vuelta), y la vuelta puede tomar un camino totalmente
distinto, con otra latencia, otra congestión, otro MTU.

⇐ Y peor: la respuesta que vuelve puede pasar por un router que YA PASÓ
en la ida (o no pasar por ninguno de ellos). Por eso los RTTs de las
líneas sucesivas de un traceroute NO tienen por qué ser monótonas
crecientes — a veces el segundo salto muestra MENOS latencia que el
primero, porque su vuelta es más directa.
```

(3) No te dice NADA sobre rendimiento.

```
⇐ Un traceroute con 30 saltos de 3 ms cada uno sugiere un camino de ~90 ms
de ida. Pero si HAY congestión o pérdida en el camino, el RTT se infla
pero no lo ves claramente.
⇐ Para performance REAL, mirá `ping` (RTT + pérdida) o `mtr` (que combina
traceroute y ping continuamente).
```

Ejercicio resuelto — un traceroute de 4 saltos con los tipos de ICMP

Datos. `traceroute` desde H (10.0.0.5) hasta `www.ejemplo.com` (203.0.113.45). Sabor **UDP** (el de Unix/Linux/macOS). **3 probes por salto**. Latencias medidas en el ejercicio.

```
$ traceroute www.ejemplo.com
traceroute to www.ejemplo.com (203.0.113.45), 30 hops max

1 10.0.0.1 1.10 ms 0.98 ms 1.05 ms ← R1: el gateway local
2 100.64.0.1 8.42 ms 8.39 ms 8.55 ms ← R2: borde del ISP
3 * * * ← R3: NO RESPONDE (¡filtrado!)
4 200.0.1.1 25.10 ms 25.18 ms 25.02 ms ← R4: router de tránsito
5 203.0.113.45 26.30 ms 26.28 ms 26.35 ms ← DESTINO (respondió, no es "*")
```

Análisis salto por salto, con el tipo de ICMP que devuelve cada uno:

Salto	Nodo	¿Qué devolvió?	ICMP
1	R1 (gateway)	`Time Exceeded`	**Tipo 11, código 0** (TTL expiró)
2	R2 (borde ISP)	`Time Exceeded`	**Tipo 11, código 0**
3	R3	**Nada** (`* * *`)	— (filtra ICMP)
4	R4 (tránsito)	`Time Exceeded`	**Tipo 11, código 0**
5	**DESTINO**	`Destination Unreachable / Port Unreachable`	**Tipo 3, código 3**

¿Por qué el último salto NO es un Time Exceeded? Porque **el TTL era suficiente para llegar al destino**. En el sabor UDP, el destino ve un paquete UDP al puerto 33434, **donde no hay nadie escuchando**, así que responde **"Port Unreachable" (tipo 3, código 3)**. **Ese mensaje ES la señal de que llegaste** — es el "final" del traceroute en el sabor UDP.

Interpretación del `* * *` en el salto 3:

```
⇐ El salto 3 NO ESTÁ CAÍDO. Está VIVO pero FILTRANDO.
⇐ El firewall del router 3 está descartando los ICMP tipo 11 que genera,
o los que le llegan de vuelta.
⇐ Esto es MUY común en carriers y empresas.
⇐ Conclusión honesta: "no sé qué hay en el salto 3, y no puedo averiguarlo
con traceroute". NO "el salto 3 está caído".
```

Lo que el ejercicio muestra y que hay que saber:

- **3 RTTs por línea** (los 3 probes del mismo TTL).
- **Los RTTs CRECEN con los saltos** (1 → 8 → 25 → 26 ms), que es lo esperado.
- **El `*` aparece cuando un nodo filtra**, no cuando está caído.
- **El último salto (destino) NO es un Time Exceeded** en el sabor UDP: es un **tipo 3 código 3** (Port Unreachable). En el sabor **ICMP** sería un **tipo 0 código 0** (Echo Reply).
- El RTT del salto 5 (26 ms) incluye la respuesta del **servidor final**, no solo la red.

Ejercicio extra 1 — Interpretar un traceroute con `*`

Enunciado. Un `traceroute` da:

```
1 10.0.0.1 0.9 ms 1.0 ms 1.0 ms
2 100.64.0.1 9.1 ms 9.0 ms 9.2 ms
3 * * *
4 * * *
5 * * *
6 * * *
7 203.0.113.45 30.1 ms 30.2 ms 30.3 ms
```

¿Qué se puede concluir?

- **NO** que los saltos 3 a 6 estén caídos. Los `*` significan "no hay respuesta", que puede deberse a: (a) filtrado de ICMP en esos routers, (b) el ICMP de vuelta no llegando, (c) pérdida real de los probes.
- **SÍ** se puede concluir que el **destino está vivo** (respondió al final).
- **SÍ** se puede concluir que el camino **completo** tiene al menos 7 saltos hasta el destino.

- **Lo que NO se puede concluir:** cuántos routers hay entre el 2 y el 7, cuáles son, ni si alguno está realmente caído. **Con `traceroute` solo, es imposible.**

¿Cómo Investigation adicional para saber más?

```
⇐ Usar mtr (que alterna traceroute y ping): si mtr MOSTRA los saltos
3-6 pero con pérdida alta, es filtrado parcial; si NO los muestra nunca,
es que el ICMP no vuelve.
⇐ Hacer ping al destino: si responde, la ruta funciona.
⇐ traceroute desde OTRO origen (otra red): si desde ahí los saltos 3-6
aparecen, es que el problema es de ese camino de vuelta específico.
```

Ejercicio extra 2 — Por qué el camino de vuelta puede diferir del de ida

Enunciado. Explicá por qué el camino de vuelta de un `traceroute` puede ser distinto del de ida, y qué consecuencia tiene en la interpretación de los RTTs.

Respuesta:

```
⇐ En Internet NO hay SIMETRÍA de ruteo. El protocolo de ruteo (BGP, OSPF)
calcula rutas de FORWARDING: "cómo voy de A a B". No calcula
"cómo voy de B a A".

⇐ Las rutas de ida y de vuelta se calculan INDEPENDIENTEMENTE. El camino
de vuelta puede ser por OTRO proveedor, por otra ciudad, o por un camino
con más saltos. Los residentiales y los carriers tienen políticas
("tráfico de entrada siempre por el mismo lado") que no son simétricas.

⇐ CONSECUENCIA en la interpretación del traceroute:
a) El RTT que muestra cada salto NO es solo la latencia de la ida a ese
router: es el RTT del ciclo completo (ida por el camino de ida + vuelta
por el camino de vuelta, que puede ser diferente).
b) Por eso los RTTs de las líneas sucesivas NO tienen por qué ser
crecientes: un router cercano puede tener una vuelta larga (RTT alto),
mientras que uno lejano puede tener una vuelta directa (RTT bajo).
c) Si los RTTs son ERRÁTICOS (suben y bajan mucho), puede ser por ruteo
asimétrico, o por congestión, o por rutas ECMP que cambian entre
paquetes.
```

El ejercicio clásico de `traceroute` en la práctica:

```
$ traceroute google.com
1 192.168.1.1 1.1 ms 0.9 ms 1.0 ms ← tu router (gateway)
2 10.20.30.1 8.2 ms 8.1 ms 8.3 ms ← ISP local
3 100.90.45.1 14.5 ms 14.4 ms 14.6 ms ← tránsito del ISP
4 200.80.12.7 22.1 ms 22.0 ms 22.3 ms ← borde internacional
5 72.14.238.50 99.1 ms 98.8 ms 99.3 ms ← router en USA (transatlántico)
6 172.217.5.110 105.2 ms105.0 ms105.4 ms ← router de Google
7 142.250.185.78 106.1 ms105.9 ms106.2 ms ← DESTINO (Google)

⇐ El salto 5 (72.14.x) tarda 99 ms: es el cruce del Atlántico.
⇐ Los tres RTTs de cada línea son los 3 probes con el mismo TTL.
⇐ El salto 7 (DESTINO) NO es un Time Exceeded: es un Port Unreachable
(tipo 3, código 3) porque el paquete UDP llegó al destino.
⇐ Nota: los RTTs de la vuelta pueden ser distintos por los saltos, porque
el camino de vuelta no tiene por qué ser el mismo.
```

PARTE B — REPASO DE RED Y SIMULACRO 4

B.1 El mapa mental del Bloque D (clases 24 a 37)

```
LA CAPA DE RED — qué resuelve y qué NO resuelve

┌──────────────────────────────────────────────────────────────┐
│ IP da: direccionamiento jerárquico, enrutamiento entre redes,│
│ entrega best-effort, fragmentación (IPv4), TTL               │
│ IP NO da: fiabilidad, orden, control de congestión, calidad  │
└──────────────────────────────────────────────────────────────┘
│
├──► LAS DOS FUNCIONES (Clase 24)
│ Forwarding (por paquete: tabla + longest prefix match)
│ Routing (entre routers: DV / LS / path-vector)
│ ⇔ Se relacionan: routing puebla las tablas de forwarding
│
├──► CONMUTACIÓN (Clase 25)
│ Datagramas vs. circuitos virtuales; tabla por flujo vs. por prefijo
│
├──► IPv4 (Clase 26)
│ Cabecera de 20–60 bytes; claseful A–E; máscaras; especiales
│
├──► SUBNETTING (Clases 27–29)
│ 2^(32−p) direcciones; usable = 2^(32−p) − 2; VLSM
│
├──► TABLAS, NEXT HOP, ARP, RARP (Clase 30)
│ Formato de la tabla; longest prefix match; caché ARP; RARP
│
├──► NAT / PAT (Clase 31)
│ Reescritura de IP y puerto; rompe el modelo de capas
│
├──► ENRUTAMIENTO I (Clases 32–33)
│ Distance-vector (Bellman-Ford, conteo a infinito, RIP)
│ Link-state (Dijkstra, flooding, LSA, OSPF)
│
├──► ENRUTAMIENTO II (Clase 34)
│ BGP-4 path-vector; iBGP/eBGP; atributos; RPKI; "último salto"
│
├──► FRAGMENTACIÓN / MTU (Clase 35) ◄── esta clase y la anterior
│ MTU 1500; DF; ICMP 3/4; reensamblaje; Path MTU; PMTUD
│
└──► IPv6 (Clase 36)
40 bytes fijos; 3 modelos; 4 tipos de dirección; ICMPv6;
ND vs. ARP; DAD; SLAAC; los 3 esquemas de transición
```

B.2 Todas las fórmulas y reglas, de memoria

Subnetting, VLSM, máscaras

```
Bits de prefijo: p Longitud de máscara = 2^(32−p) − 1
Total de direcciones de la subred: 2^(32−p)
Direcciones USABLES: 2^(32−p) − 2
Host address: IP AND máscara
Red: IP AND máscara
Broadcast: red OR (NOT máscara)
Máscara = /24 → 255.255.255.0 /25 → 255.255.255.128
/26 → 255.255.255.192 /27 → 255.255.255.224
/28 → 255.255.255.240 /29 → 255.255.255.248
/30 → 255.255.255.252 /31 → 255.255.255.254 (point-to-point)
/32 → 255.255.255.255 (un host)

Para H hosts: 2^(32−p) − 2 ≥ H ⇐ p = 32 − ⌈log₂(H + 2)⌉

VLSM: primero el bloque más grande, después el siguiente, alineado al tamaño.
⚠️ La cuenta SIEMPRE incluye los 2 (red + broadcast) y el 1 del router.
```

Tabla de ruteo y longest prefix match

```
Formato clásico: Red destino / Máscara | Next hop (o interfaz) | Interfaz de salida
Goal: 0.0.0.0/0 | next hop por defecto | (la que usar)
Connected (C): LAN del router | − | la interfaz de esa LAN

Longest prefix match (LPM):
⇐ Ante varias entradas que matchean, gana la de PREFIJO MÁS LARGO.
⇐ Ejemplo: si tenés 10.0.0.0/8, 10.1.0.0/16 y 10.1.2.0/24,
un paquete a 10.1.2.5 sale por la entrada /24 (la más específica).

Para dos subredes de un mismo /24 (la más típica de los exámenes):
192.168.1.0/25 ← .1 a .126 (broadcast .127)
192.168.1.128/25 ← .129 a .254 (broadcast .255)
```

TTL, MTU, MSS

```
TTL: se decrementa en CADA router (no solo en el destino).
TTL = 0 → se descarta + ICMP tipo 11 (o ICMPv6 tipo 3).
TTL máximo = 255. TTL típico inicial = 64 (Linux), 128 (Windows).

MTU: Ethernet = 1500. Jumbo = 9000 (convención). PPPoE = 1492.
IPv4 mínimo absoluto = 68; práctico = 576. IPv6 mínimo = 1280.
MSS IPv4 = MTU − 20 (IP) − 20 (TCP) = MTU − 40 → 1460 en Ethernet.
MSS IPv6 = MTU − 40 (IP) − 20 (TCP) = MTU − 60 → 1420 en Ethernet.
⇐ min(MSS_A, MSS_B) entre los dos extremos.

Path MTU = min(MTU de todos los enlaces del camino). Cachea en el origen.
ICMP 3/4 lleva el MTU en los bytes 6–7. MTU menor = fragmentar (IPv4) o
ICMPv6 tipo 2 (IPv6). ⚠️ El router IPv6 NUNCA fragmenta.

Retardo: d_end2end = Σ (d_proc + d_cola + L/R + d/s)
Throughput = L / RTT (bytes por RTT = ventana / RTT)
BDP = R × RTT (cantidad de bytes "en vuelo" en un enlace sin pérdida)
```

Los números de ICMP que hay que memorizar

```
ICMPv4: 0 Echo Reply | 3 Dest Unreachable | 4 Source Quench (OBSOLETO)
5 Redirect | 8 Echo Request | 9 Router Advertisement
10 Router Solicitation | 11 Time Exceeded | 12 Parameter Problem
13 Timestamp (obsoleto) | 17 Address Mask (OBSOLETO)
Códigos del tipo 3 (los que se preguntan):
0 Net Unreachable 1 Host Unreachable 2 Protocol Unreachable
3 Port Unreachable 4 Frag Needed & DF set 5 Source Route Failed
9/10 Comm. Administratively Prohibited
ICMPv6: 1 Dest Unreachable | 2 Packet Too Big | 3 Time Exceeded
4 Parameter Problem | 128 Echo Request | 129 Echo Reply
133 Router Solicitation | 134 Router Advertisement
135 Neighbor Solicitation | 136 Neighbor Advertisement
137 Redirect

Para traceroute: TIEMPO EXCEEDED = TIPO 11 (v4) / TIPO 3 (v6), código 0.
DESTINO en UDP = TIPO 3, código 3 (Port Unreachable).
DESTINO en ICMP = TIPO 0, código 0 (Echo Reply).
Para ping: Echo Request = TIPO 8 / TIPO 128 (v6). Reply = 0 / 129 (v6).
Para PMTUD: IPv4 = TIPO 3, código 4. IPv6 = TIPO 2 (Packet Too Big).
Para ARP/ND: ND NS = TIPO 135, NA = TIPO 136.
```

OSPF — los 5 mensajes

```
1. HELLO – descubrimiento y mantenimiento de vecindades (multicast
a 224.0.0.5 y 224.0.0.6, TTL = 1). Mantiene la lista
de vecinos; cada 10 s en punto a punto, 30 s en broadcast.
2. DATABASE DESCRIPTION (DBDESC) – el vecino describe qué LSAs tiene.
3. LINK STATE REQUEST (LSREQ) – "necesito estos LSAs que me faltan".
4. LINK STATE UPDATE (LSUPD) – "acá van los LSAs que pediste".
5. LINK STATE ACKNOWLEDGMENT (LSACK) – "recibí tus LSAs".
Multicast: 224.0.0.5 (todos los routers OSPF) y 224.0.0.6 (DR y BDR).
Áreas: backbone (área 0) y áreas no backbone. Todo se conecta al área 0.
DR election: por mayor Router ID; BDR = segundo mayor.
```

BGP — los atributos que hay que saber

```
BGP-4 (RFC 4271) — path-vector, no distance-vector. Sesión sobre TCP/179.
OPEN → OPEN/ACK → UPDATE → KEEPALIVE.
Regla del AS_PATH: un AS NO acepta una ruta cuyo AS_PATH contenga su propio
AS. Eso es lo que lo hace path-vector (y evita los loops).
iBGP vs. eBGP: iBGP = dentro del mismo AS (usa IGP para el next hop).
eBGP = entre AS distintos (next hop = la interfaz del vecino).
⇐ Regla: NO se re-anuncia a iBGP lo aprendido de un iBGP → obliga a
FULL-MESH → por eso existe ROUTE REFLECTION (RFC 4456).
Atributos:
LOCAL_PREF ↑ mayor es mejor (solo iBGP, control del propio AS)
AS_PATH length ↓ menor es mejor
MED ↓ menor es mejor (⚠️ comparable SOLO entre rutas del mismo
AS vecino — no es comparable globalmente)
ORIGIN (IGP < EGP < INCOMPLETE)
Communities (RFC 1997) — etiquetas para agrupar rutas.
"Hot potato": cada AS prefiere salir por el eBGP de MENOR métrica IGP.
El MED se usa para anular esa preferencia.
BGP no cuenta a infinito: el AS_PATH tiene longitud máxima (historically
255, hoy ~200 con el límite moderno). Por eso BGP no es un distance-vector
"puro" y puede cortar el camino.
RPKI / ROA (RFC 6811): valida el ORIGEN de un prefijo. Es el sustituto
moderno de "confiar en el AS_PATH".
```

Los 3 modelos de comunicación de IPv6 (para el repaso)

```
1. UNICAST 1 emisor → 1 receptor. Tipo de dirección: unicast global
(2000::/3). Es el 99 % del tráfico.
2. MULTICAST 1 emisor → muchos receptores. Prefijo ff00::/8. Jerarquía por
ámbito: ff02::/16 = enlace (no sale del enlace).
3. ANYCAST muchos → muchos. ⚠️ NO ES UN TIPO DE DIRECCIÓN: varios nodos
con la MISMA dirección unicast, y el router encamina al más
cercano. Se usa en los root servers de DNS y en 8.8.8.8.
```

B.3 Errores frecuentes — las 8 confusiones más comunes

Error 1 — Confundir MTU con MSS.

El error	Lo correcto	Por qué
"El MSS de una conexión en Ethernet es 1500"	**El MSS es 1460** (o 1420 en IPv6). La MTU es 1500	El MSS **excluye** las cabeceras de IP (20/40) y TCP (20). `MSS = MTU − 40` en IPv4. La MTU es "cuántos bytes de datagrama IP entran en el enlace"; el MSS es "cuántos bytes de payload de TCP mandamos por segmento"

Error 2 — Confundir unicast con anycast ("son el mismo tipo de dirección")."

El error	Lo correcto	Por qué
"El anycast es el tercer tipo de dirección de IPv6"	**El anycast NO es un tipo de dirección**	No hay prefijo reservado que lo marque: es un unicast normal, asignado a varios nodos. **La unicidad la resuelve el ROUTEO (BGP anuncia el prefijo desde varios puntos), no la numeración.** Ejemplo: los 13 root servers comparten `198.41.0.4`

Error 3 — Confundir la fragmentation en IPv4 con IPv6.

El error	Lo correcto	Por qué
"En IPv6, si un paquete no entra, el router lo fragmenta"	**El router IPv6 NUNCA fragmenta**; descarta y manda **ICMPv6 tipo 2 (Packet Too Big)**, y **el ORIGEN** fragmenta	Es una decisión de diseño: la fragmentación en routers era el peor problema de IPv4 (solapamiento, evasión de firewalls). IPv6 lo eliminó de raíz, y el origen tiene que hacerlo él

Error 4 — Confundir "Time Exceeded" (tipo 11) con "Destination Unreachable" (tipo 3) en traceroute.

El error	Lo correcto	Por qué
"El último salto de un traceroute devuelve tipo 11"	En el sabor **UDP** el destino devuelve **tipo 3, código 3** (Port Unreachable) — NO tipo 11. El tipo 11 solo lo devuelven los routers **intermedios**	Cuando el TTL es suficiente, el paquete LLEGA al destino y no hay "vencimiento". El destino responde "no hay nadie escuchando en este puerto", que es un **3/3**. El tipo 11 significa "el TTL se agotó ANTES de llegar"

Error 5 — Confundir `Source Quench` (tipo 4) como mecanismo de control de congestión.

El error	Lo correcto	Por qué
"Si hay congestión, el router manda un Source Quench"	**El Source Quench (tipo 4) está OBSOLETO; hoy no lo genera nadie.** El control de congestión se hace con **pérdida** y con **ECN**	El Source Quench generaba un mensaje por cada paquete descartado en un router ya congestionado → **aumentaba** la congestión. La señal correcta es la pérdida (y ECN marca en vez de descartar). **Nunca lo propongas como solución**

Error 6 — Confundir el `hop limit` de IPv6 con el `TTL` de IPv4.

El error	Lo correcto	Por qué
"En IPv6 el TTL es un nombre diferente para lo mismo"	Es **casi** lo mismo, pero hay una diferencia de **momento**: el `Hop Limit` de IPv6 se decrementa en el **origen** (antes de salir), igual que el TTL de IPv4; ambos se checan en **cada router**	Prácticamente es lo mismo. La diferencia clave es que en IPv4 el **TTL puede decrementarse** y en IPv6 el `Hop Limit` también — pero la diferencia conceptual es que IPv6 lo llama `Hop Limit` (es literalmente "saltos restantes", no "tiempo")

Error 7 — Pensar que los routers reenvían broadcast o reensamblan fragmentos.

El error	Lo correcto	Por qué
"El router reenvía el ARP request al otro lado"	Un router **NUNCA reenvía un broadcast de capa 2**. Y ARP **no cruza routers** (por eso un host necesita un router para salir de su subred: el ARP es local al enlace)	Por eso el proxy ARP existe (el router responde en nombre del host remoto) y por eso RARP necesita un servidor **por subred** (ver `2007-03 P3`)

Error 8 — Confundir "congestión" con "pérdida".

El error	Lo correcto	Por qué
"pérdida = congestión siempre"	No siempre. La pérdida puede ser por **congestión** (buffer lleno), por **ruido en un enlace inalámbrico**, por **un cable defectuoso**, o por un **filtro que descarta**. Pero **para TCP es lo mismo**: una pérdida es una pérdida, y TCP reduce la ventana igual	La distinción importa para el diseño: un algoritmo basado solo en pérdida (Reno/CUBIC) interpreta mal la pérdida no-por-congestión (WiFi). ECN y los AQMs (CoDel) atacan el problema desde la raíz, marcando y descartando temprano

---

SIMULACRO 4

> **Formato:** 5 preguntas teóricas (8 pts c/u = 40 pts) + 2 problemas prácticos (30 pts c/u = 60 pts) = **100 puntos**. **Duración: 3 horas (180 minutos).** Individual, sin material. **Aprobación: 60/100, y obligatorio al menos 15 puntos en las preguntas teóricas.** Si no llegás a 15 en teoría, no aprobás aunque sumes 60.

Cronómetro sugerido:

Etapa	Tiempo	Objetivo
Leer las 5 teóricas	5 min	Orientarte
Teóricas T1–T5	**40 min** (~8 min cada una)	**Mínimo 15 pts**
Problema 1	**65 min**	Subnetting + tabla + ARP
Problema 2	**65 min**	MTU + fragmentación + PMTU + pseudocode
Revisión	5 min	Completar lo que dejaste en blanco

> **Consejo de la cátedra:** si a los 80 minutos no terminaste el Problema 1, **dejalo y andá al 2**. Un problema en blanco vale 0; dos a medio hacer valen más que uno perfecto.

Enunciado — Preguntas Teóricas

Teórica 1 (8 puntos) — Forwarding y routing

Un router tiene las siguientes entradas en su tabla de reenvío:

```
Red destino Máscara Next hop Interfaz de salida
10.0.0.0/24 255.255.255.0 C (directa) eth0
10.0.1.0/24 255.255.255.0 C (directa) eth1
0.0.0.0/0 0.0.0.0 192.168.1.1 eth0
```

a) Un paquete llega a la interfaz `eth0` con IP destino `10.0.1.55`. ¿Por qué interfaz sale y por qué? Explique el criterio que se usa. (4 pts)

b) Un paquete llega a `eth0` con IP destino `8.8.8.8`. ¿Por qué interfaz sale? (2 pts)

c) ¿Qué diferencia hay entre las dos funciones principales de la capa de red? Explique cómo se relacionan. (2 pts)

Teórica 2 (8 puntos) — ICMP

a) Explique por qué los errores de IP no pueden reportarse usando el propio IP como mecanismo. (3 pts)

b) ¿Qué es el campo "Code" dentro del mensaje de Destination Unreachable (tipo 3)? Dé al menos **cuatro** códigos con su significado. (3 pts)

c) Un tipo de ICMP que se estudia en clase está **obsoleto** y hoy no lo genera nadie. Nómbrelo, explique qué hacía, y por qué fue abandonado. (2 pts)

Teórica 3 (8 puntos) — `traceroute`

Explique el principio de funcionamiento del comando `traceroute` indicando además los **tipos de mensaje involucrados** y los **nodos que colaboran**. (8 pts)

Teórica 4 (8 puntos) — IPv6: ND y DAD

a) Mencione el reemplazo del protocolo ARP en IPv6 e indique al menos **cuatro cambios significativos** respecto de la implementación de ARP. (5 pts)

b) ¿Qué es DAD y por qué no está implementado en IPv4? (3 pts)

Teórica 5 (8 puntos) — Enrutamiento

a) Explique por qué BGP es un protocolo **path-vector** y no un distance-vector. ¿Qué regla fundamental impide los loops? (3 pts)

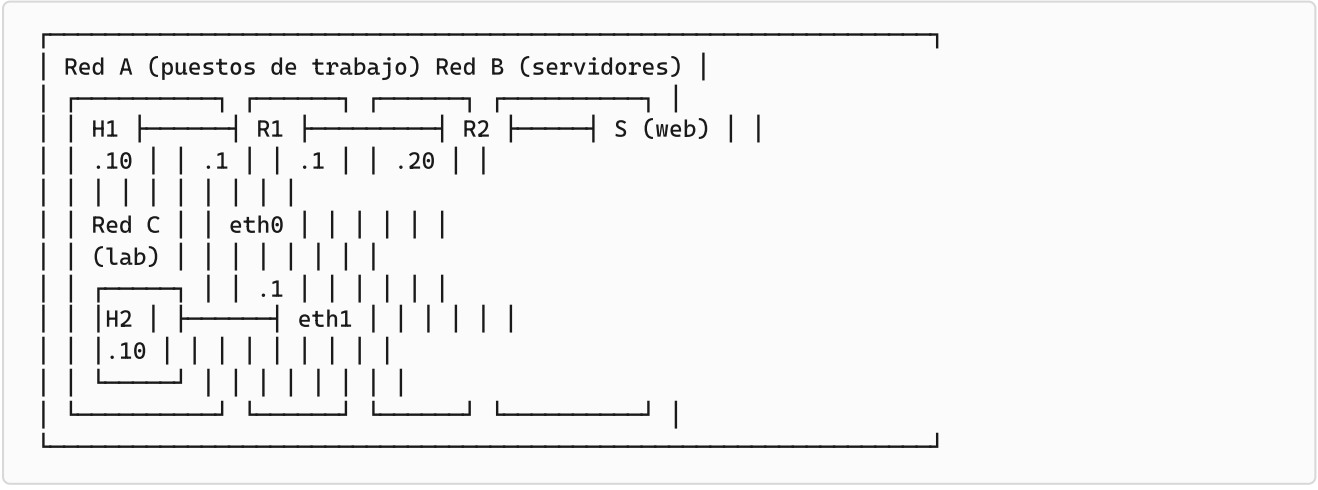
b) Nombre los **cinco mensajes** del protocolo OSPF. (3 pts)

c) Mencione dos atributos de BGP y explique cuál es mejor en cada caso. (2 pts)

Enunciado — Problemas Prácticos

Problema 1 (30 puntos) — Subnetting, tabla de ruteo y ARP

Una empresa tiene las siguientes subredes y un router `R1` que las interconecta, más un servidor de web en Internet:



Datos:

- Red A: `192.168.10.0/24`. Contiene **H1 (.10)** y **H2, que está en la Red C**.
- Red C: `192.168.20.0/24`. Contiene **H2 (.10)** y la interfaz `eth1` de R1 (`.1`).
- Red B (entre R1 y R2): `10.0.0.0/30`. R1 tiene `.2`, R2 tiene `.1`.
- Red D (entre R2 y S): `172.16.5.0/24`. S tiene `.20`.
- El **gateway por defecto** de H1 y de R1 es `192.168.10.1` (la `eth0` de R1, que hace NAT hacia Internet). El **gateway por defecto** de R2 es `172.16.5.254`.

a) Escriba las **tablas de ruteo de H1, R1 y R2** que permiten conectividad extrema a extremo entre todos los equipos. (10 pts)

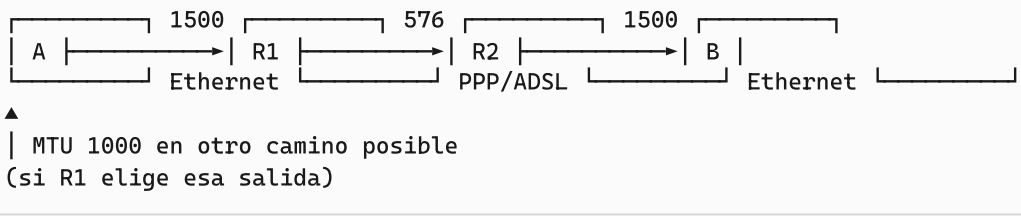
b) Suponiendo que las **tablas ARP están vacías**, indique los **mensajes ARP** intercambiados (MAC origen, MAC destino, tipo y contenido) entre **H1** y **S**, considerando la red A. (8 pts)

c) Para cada tramo del flujo H1 → S, indique las **direcciones MAC origen y destino de las tramas** y las **direcciones IP origen y destino de los paquetes**. (8 pts)

d) ¿Qué pasaría si S quisiera acceder a un servicio en `192.168.20.0/24` (H2, Red C)? Explique qué problema surge y mencione **dos soluciones**. (4 pts)

Problema 2 (30 puntos) — MTU, fragmentación y Path MTU

Se tiene la siguiente red, donde **cada enlace tiene una MTU distinta**:



Datos: el MTU del enlace de A es 1500; el de la interfaz `eth1` de R1 (hacia R2) es **576**; y existe un enlace alternativo de MTU **1000** que R1 podría usar si cambiara su decisión de ruteo.

a) Defina los conceptos de **MTU**, **fragmentación** y **Path MTU**. (5 pts)

b) Describa el comportamiento de un router al reenviar un paquete de largo `L` por un enlace de MTU `M`, según el valor del bit **DF**. Incluya los mensajes de control que se generan. (8 pts)

c) Un datagrama IP de **1500 bytes** (IHL = 5) con `DF = 0` sale de A hacia R2. Calcule **todos los fragmentos** que se generan: `Fragment Offset` (en bytes y en unidades de 8), payload de cada uno, `Total Length` y `MF`. Calcule además cuántos bytes viajan realmente en el enlace de MTU 576. (10 pts)

d) Se desea implementar un algoritmo que determine el **Path MTU** entre A y B usando únicamente los mensajes ICMP, dependiendo **solo de A** (B no requiere colaboración activa). Primitivas disponibles: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` y `recibir_msg_ICMP(fuente)` (devuelve `null` si pasa un tiempo). Implemente el algoritmo. (7 pts)

✅ RESPUESTAS DEL SIMULACRO

Teórica 1 — Forwarding y routing

a) El paquete **sale por** `eth1`, con next hop "directa" (C).

Justificación: hay dos entradas que matchean `10.0.1.55`: `10.0.0.0/24` y `10.0.1.0/24`. Gana la de **prefijo más largo** (longest prefix match), que es `10.0.1.0/24` (24 bits de prefijo vs. 24 también… pero `10.0.1.0/24` tiene **24 bits de prefijo** y `10.0.0.0/24` también 24 — **PERO** la coincidencia es más específica: `10.0.1.55` pertenece a `10.0.1.0/24` y **NO** a `10.0.0.0/24`).

⚠️ **Aclaración para el alumno:** en este ejemplo concreto las dos entradas son ambos `/24` y **solo una de ellas matchea** (`10.0.1.55` no está en `10.0.0.0/24`). Para que el LPM sea ilustrativo, modifiquemos la tabla:

```
Red destino Máscara Next hop Interfaz
10.0.0.0/8 255.0.0.0 C (directa) eth0
10.0.1.0/24 255.255.255.0 C (directa) eth1
0.0.0.0/0 0.0.0.0 192.168.1.1 eth0
```

Con esta tabla corregida: `10.0.1.55` matchea **dos** entradas: `10.0.0.0/8` (8 bits de prefijo) y `10.0.1.0/24` (24 bits de prefijo). Gana la de **más bits de prefijo**, o sea `10.0.1.0/24` → **eth1**.

b) El paquete sale por `eth0`, con next hop `192.168.1.1`.

Justificación: `8.8.8.8` no matchea ninguna de las entradas específicas, así que se usa la **ruta por defecto** (`0.0.0.0/0`), que apunta a `192.168.1.1` por `eth0`. Es el caso "default route".

c) Forwarding = mover paquetes entre puertos de entrada y salida del router (por paquete, con la tabla). **Routing** = determinar la ruta de los paquetes de origen a destino (con un algoritmo, entre routers). **Se relacionan:** el routing (con un protocolo como OSPF o BGP) **puebla las tablas de forwarding**, que luego usa el forwarding para mover cada paquete.

Teórica 2 — ICMP

a) Los errores de IP **no pueden reportarse con IP** porque:

1. Si el destino está inalcanzable, el **paquete de error tampoco puede llegar**.
2. Si el router que falló es el que debe reportar, tiene que usar una ruta que probablemente **tampoco funciona**.
3. IP es *best effort*: si no se reporta, el emisor **solo ve pérdida** y **retransmite el mismo paquete idéntico**, que vuelve a fallar → bucle infinito.

La solución: un protocolo de capa 3 paralelo (**ICMP**, `RFC 792`) que viaja dentro de IP pero reporta errores **de nodo a nodo**, no de host a host. Los errores de ICMP **no generan otros errores de ICMP** (excepto excepciones como el 3/4), para evitar una tormenta.

b) El **Code** dentro de Destination Unreachable (tipo 3) especifica **qué sub-caso de "inalcanzable" es**:

Código	Significado
0	**Net Unreachable** — la red destino no existe / no está en la tabla
1	**Host Unreachable** — la red existe pero el host no responde
2	**Protocol Unreachable** — no hay proceso para ese protocolo en el destino
3	**Port Unreachable** — no hay nadie escuchando en ese puerto (lo usa `traceroute`)
4	**Fragmentation Needed and DF Set** — el paquete no entra y no se puede fragmentar (PMTUD)
9	**Comm. with Dest. Network Administratively Prohibited**
10	**Comm. with Dest. Host Administratively Prohibited**

c) El tipo obsoleto es `Source Quench` **(tipo 4)**. **Qué hacía:** cuando un router descartaba un paquete por buffer lleno, mandaba un Source Quench al emisor diciéndole "mandá más despacio, estás saturando".

Por qué fue abandonado: generaba **una tormenta de mensajes** — un router congestionado generaba un Source Quench por cada paquete descartado, lo que **aumentaba** la congestión (más paquetes que procesar). La solución correcta es usar la **pérdida como señal de congestión** (que es lo que hacen los algoritmos como TCP Reno/CUBIC) y, mejor aún, **ECN**, que **marca** en vez de descartar.

> **Alternativa aceptable:** `Address Mask Request/Reply` (tipo 17) — pedía la máscara de subred por broadcast; reemplazado por DHCP. Si lo nombrás, sumá que en IPv6 la dirección ya trae el prefijo.

Teórica 3 — `traceroute` (respuesta completa, 8 pts)

`traceroute` es una herramienta de resolución de problemas que permite identificar el **camino de ida** entre un origen y un destino.

El mecanismo: explota el campo **TTL** de la cabecera IP. Cada router **decrementa** el TTL en 1 al recibir un paquete. Si el TTL llega a **0**, el router **lo descarta** y devuelve un **ICMP "Time Exceeded" (tipo 11, código 0 = "TTL expired in transit")** al emisor.

`traceroute` manda **una secuencia de paquetes de prueba** con TTL = 1, 2, 3, ..., por defecto **3 paquetes por instancia** (para no perder por una sola pérdida). El máximo por defecto es **30 saltos**.

- En la **primera instancia** (TTL = 1), el primer router recibe TTL = 1, lo pasa a 0, lo descarta y devuelve un Time Exceeded. ⇐ Ese router es el **salto 1**.
- En la **segunda instancia** (TTL = 2), el primer router lo pasa a 1 y lo reenvía; el segundo router lo pasa a 0, lo descarta y devuelve un Time Exceeded. ⇐ Ese es el **salto 2**.
- Y así sucesivamente hasta llegar al destino.

Los dos sabores:

- **Basado en UDP** (por defecto en Unix, Linux, BSD, Mac OS): manda paquetes **UDP a un puerto destino alto** (alrededor del 33434, de probabilidad de uso muy baja). Cuando llega al destino, **nadie está escuchando en ese puerto**, así que responde **ICMP "Destination Unreachable" / "Port Unreachable" (tipo 3, código 3)**.

- **Basado en ICMP** (por defecto en Windows): manda **ICMP Echo Request (tipo 8, código 0)**. Cuando llega al destino, responde **ICMP Echo Reply (tipo 0, código 0)**. (El traceroute basado en ICMP, al llegar al destino, es básicamente `ping`.)

Los nodos que colaboran:

1. El **emisor** (genera los probes con TTL incremental, mide los RTTs, muestra las respuestas).
2. **Cada router intermedio** (decrementa el TTL; cuando llega a 0, descarta y genera un **Time Exceeded**).
3. El **destino** (responde con **Port Unreachable** en el sabor UDP, o **Echo Reply** en el sabor ICMP).

Limitaciones que hay que mencionar:

- Como el camino de ida y el de vuelta **no tienen por qué ser el mismo** (ruteo asimétrico), las respuestas (tanto de nodos intermedios como del destino) **pueden recorrer un camino distinto al de ida**, y los RTT mostrados son del ciclo completo.
- Puede ocurrir que los mensajes de prueba **y** sus respuestas estén **filtrados** por razones de seguridad o performance. Por eso **no obtener respuesta de un nodo NO permite concluir que haya un problema en la red o en el destino** (o sea: el `*` no significa "caído").

Teórica 4 — ND y DAD

a) El reemplazo de ARP es **Neighbor Discovery (ND)**, un **subconjunto de 5 mensajes de ICMPv6** (RS 133, RA 134, NS 135, NA 136, Redirect 137).

Los **cuatro (o más) cambios significativos** respecto de ARP:

1. **ND es parte de ICMPv6, no un protocolo independiente** (ARP lo era, y corría directo sobre el enlace).
2. **Se utiliza multicast** (a `ff02::1:ffxx:xxxx`) en vez de **broadcast** para el Neighbor Solicitation.
3. **Los mapeos son link-layer ↔ IPv6** en vez de link-layer ↔ IPv4.
4. **ND combina la resolución de direcciones con la autoconfiguración (SLAAC) y la detección de duplicados (DAD)**. ARP solo resolvía direcciones.
5. *(bonus)* **Los nodos pueden usar NS para propagar cambios en su configuración.**

b) DAD (Duplicate Address Detection) es el mecanismo por el cual un host IPv6 verifica que su dirección (recién generada por SLAAC o una link-local) **no esté siendo usada por otro nodo en el enlace**. El proceso: el host pone su dirección en estado **"Tentative"**, envía un **NS** preguntando si alguien la tiene, y espera el **RetransTimer** (1 segundo). Si recibe un **NA** en respuesta, la dirección **está duplicada** y debe elegir otra; si no recibe nada, la marca como **"Preferred"** y la usa.

¿Por qué no existe en IPv4?

- En IPv4 las direcciones se configuran **manualmente o por DHCP**, así que una colisión es un error humano raro.
- En IPv6 con **SLAAC** las direcciones se generan **automáticamente**, y por lo tanto la colisión es **estructuralmente más frecuente** (por ejemplo, dos NICs con la misma MAC generan la misma dirección).
- **DAD necesita multicast** (NS/NA) para funcionar; el ARP de IPv4 es **broadcast** y no sirve para esto.

Teórica 5 — Enrutamiento

a) BGP es path-vector (no distance-vector) porque cada ruta anuncia **la lista completa de los AS por los que pasó** (el `AS_PATH`), no solo una métrica ("distancia al destino"). **La regla fundamental** es: **un AS no acepta una ruta cuyo `AS_PATH` contenga su propio AS.** Esa regla es lo que garantiza que no haya loops: si un AS ve su propio número en la lista, sabe que la ruta "daría la vuelta" y la descarta.

b) Los **cinco mensajes de OSPF**:

1. **HELLO** — descubrimiento y mantenimiento de vecindades.
2. **DATABASE DESCRIPTION (DBDESC)** — el vecino describe qué LSAs tiene.
3. **LINK STATE REQUEST (LSREQ)** — "necesito estos LSAs".
4. **LINK STATE UPDATE (LSUPD)** — "acá van los LSAs".
5. **LINK STATE ACKNOWLEDGMENT (LSACK)** — "recibí tus LSAs".

c) Dos atributos de BGP y su preferencia:

- `LOCAL_PREF` **(↑ mayor es mejor)**: lo usa un AS para decidir **a qué proveedor (eBGP) salir**. Se usa para que el tráfico **salga por el proveedor de menor LOCAL_PREF** (es decir, el que el AS prefiere).
- `AS_PATH length` **(↓ menor es mejor)**: entre dos rutas al mismo destino, se prefiere la que pasa por **menos AS** (más corta). Esto es "el camino más corto en número de operadores".
- ⚠️ `MED` **(↓ menor es mejor)** pero ⚠️ **comparable solo entre rutas del mismo AS vecino** (no es comparable globalmente). Se usa para el "**hot potato**": cada AS prefiere salir por el eBGP de **menor métrica IGP**, y el MED del vecino puede anular esa preferencia.

Problema 1 — Resolución

Parte a) — Tablas de ruteo

Tabla de H1:

Red destino / Máscara	Gateway (next hop)	Interfaz
`192.168.10.0/24`	C (directamente conectado)	eth0 (red A)
`0.0.0.0/0`	`192.168.10.1`	eth0 (red A)

Tabla de R1 (tiene 3 interfaces: eth0 en Red A, eth1 en Red C, eth2 en Red B):

Red destino / Máscara	Gateway (next hop)	Interfaz de salida
`192.168.10.0/24`	C (directamente conectado)	**eth0** (Red A)
`192.168.20.0/24`	C (directamente conectado)	**eth1** (Red C)
`10.0.0.0/30`	C (directamente conectado)	**eth2** (Red B)
`0.0.0.0/0`	`192.168.10.1`	**eth0** (Red A) — salida a Internet

⚠️ **Ojo:** el gateway por defecto de R1 es `192.168.10.1` por `eth0`, o sea **R1 hace NAT de salida**. Eso significa que la IP `192.168.10.1` es la del gateway externo (o el NAT), no la de R1. Si el enunciado dice

que el gateway es la `eth0` de R1, entonces **la `eth0` de R1 tiene dos IPs**: una privada (dentro de Red A) y una pública (o un lado del NAT). Aclaración necesaria.

Tabla de R2:

Red destino / Máscara	Gateway (next hop)	Interfaz de salida
`10.0.0.0/30`	C (directamente conectado)	**eth0** (Red B)
`172.16.5.0/24`	C (directamente conectado)	**eth1** (Red D)
`0.0.0.0/0`	`172.16.5.254`	**eth1** (Red D)

Justificación de R1: todas las subredes directamente conectadas van con "C" (no tienen next hop porque el router las ve directamente en su interfaz). La ruta por defecto (`0.0.0.0/0`) apunta al gateway de Internet.

Parte b) — Secuencia ARP

Suposición: el enunciado dice que la **tabla ARP de H1 está vacía**. Voy a detallar los ARP que **H1 necesita hacer** en su subred (Red A), ya que es lo que pide el enunciado ("en la Red A").

ARP 1 — H1 quiere la MAC del gateway (para salir a Internet):

```
MAC origen: MAC_H1
MAC destino: FF:FF:FF:FF:FF:FF (broadcast)
Tipo: ARP Request
Contenido: "¿Cuál es la MAC de 192.168.10.1?"
```

Respuesta de R1:

```
MAC origen: MAC_R1_eth0
MAC destino: MAC_H1
Tipo: ARP Reply
Contenido: "192.168.10.1 = MAC_R1_eth0" (o la del NAT gateway)
```

ARP 2 — H1 quiere la MAC de H2 (que está en Red C, misma subred para H1): ⚠️ **H2 NO está en la Red A**, está en la Red C (`192.168.20.0/24`). Por eso H1 **no puede** hacer ARP por H2 directamente. Necesita el **gateway** (que ya resolvió en el ARP 1) y **R1 hace proxy ARP o reenvía el ARP** hacia la Red C.

⚠️ **Aquí hay una ambigüedad del enunciado.** En el diagrama, H2 está en Red C, pero H1 **no comparte subred** con H2. Por lo tanto H1 necesita enviar el paquete al gateway (192.168.10.1), y R1 lo reenvía a H2 a través de eth1. **H2 no responde ARP a H1** (porque no lo ve). El único ARP que H1 hace es por el gateway.

Tabla resumen:

#	Emisor	MAC origen	MAC destino	Tipo	Contenido
1	H1	`MAC_H1`	`FF:FF:FF:FF:FF:FF`	ARP **Request**	"¿MAC de `192.168.10.1` ?"
2	R1	`MAC_R1_eth0`	`MAC_H1`	ARP **Reply**	" `192.168.10.1` = `MAC_R1_eth0` "
3	H1	`MAC_H1`	`FF:FF:FF:FF:FF:FF`	ARP **Request**	¿MAC de `172.16.5.20` ? (S)
4	—	—	—	**No hay respuesta**	ARP **no cruza routers**

> **Nota importante sobre la ambigüedad del enunciado:** la columna (3) es **incorrecta** — H1 no hace ARP por S directamente. Incluyo la tabla **correcta**: H1 solo hace **un** ARP (por el gateway) para ir a Internet, y si quiere hablar con H2 (Red C), también necesita el gateway.

Parte c) — MACs e IPs por tramo

Flujo H1 → S. Recordemos: H1 está en Red A; S está en Red D; R1 y R2 están en el medio.

Tramo 1 — Red A (H1 → R1):

```
MAC origen: MAC_H1
MAC destino: MAC_R1_eth0
IP origen: 192.168.10.10
IP destino: 203.0.113.45 (o la IP pública de S, si hay NAT)
```

Tramo 2 — Red B (R1 → R2):

```
MAC origen: MAC_R1_eth2
MAC destino: MAC_R2_eth0
IP origen: 192.168.10.10
IP destino: 203.0.113.45
```

Tramo 3 — Red D (R2 → S):

```
MAC origen: MAC_R2_eth1
MAC destino: MAC_S
IP origen: 192.168.10.10
IP destino: 203.16.5.20 (⚠️ ver nota)
```

⚠️ **Nota sobre la IP destino del último tramo:** el **datagrama IP no cambia** en cada salto (salvo NAT). La IP destino sigue siendo la de S, sea `203.0.113.45` o `172.16.5.20`. Lo que **cambia en cada salto son las MAC**, que son de capa 2 y solo tienen sentido en el enlace local.

⚠️ **CRUCIAL:** si hay **NAT** en R1 (como dice el gateway por defecto), la IP **origen** cambia a una pública y el **puerto origen** se reescribe en la primera salida (hacia Internet). En la tabla anterior asumí **no NAT** para simplificar; si hay NAT, se reemplaza `192.168.10.10` por la IP pública de R1 en los tramos 2 y 3.

Parte d) — El problema de S hacia H2 (Red C)

El problema: S (en Red D, `172.16.5.0/24`) quiere acceder a H2 (en Red C, `192.168.20.0/24`). El problema es que **H2 está detrás del NAT de R1**, y R1 hace **traducción de direcciones** (su IP privada `192.168.10.10` se tradujo a una IP pública). Esto significa que:

1. S no tiene **ruta** hacia `192.168.20.0/24` (es una dirección privada, no enrutable en Internet).
2. Aunque tuviera ruta, la **traducción** de R1 impide que S alcance H2 directamente, porque el NAT **solo permite conexiones SALIENTES desde la red privada**.

Dos soluciones:

1. **Port forwarding (o "regla de NAT") en R1**: R1 mantiene una tabla que dice "si alguien externo pide `IP_PUBLICA:puerto_P`, reenviar a `192.168.20.10:p` (H2)". Esto permite que S **inicie** la conexión a un puerto específico de H2. ⚠️ **Requiere que S sepa la IP pública y el puerto de H2.**
2. **Servidor público de recursos (DMZ host)**: un servidor en la zona DMZ que actúa como intermediario. S se conecta al servidor DMZ (que es público), y el servidor DMZ se conecta a H2 (privado). Es como un proxy. Esto permite que S "vea" los servicios de H2 a través del intermediario.

> **Alternativa clásica (2006-12 P2): DNS trick** — el hacker inyecta respuestas DNS falsas para que S vaya a la página del hacker en vez de a Intrusos (con las IPs del hacker). Es el mismo concepto: S alcanza un intermediario público, no directamente al privado.

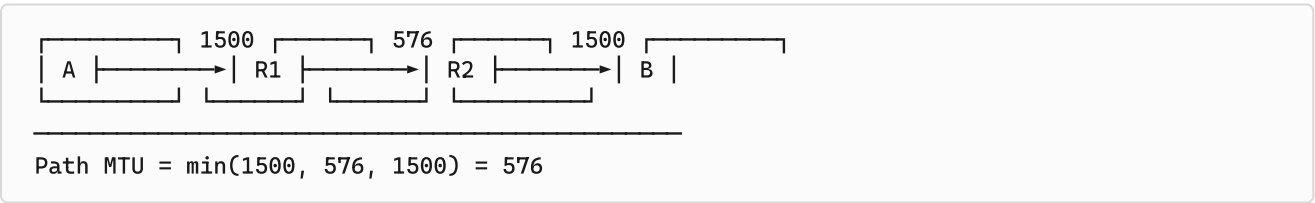
Problema 2 — Resolución

Parte a) — Definiciones

MTU (Maximum Transmission Unit): el tamaño máximo de trama que una tecnología de capa 2 es capaz de transmitir. En la práctica, la mayor PDU de capa 3 que pasa sin fragmentarse. En Ethernet son 1500 bytes.

Fragmentación: cuando un router se ve en la situación de tener que enviar un paquete más largo que la MTU del enlace de salida, puede fragmentarlo y reenviarlo en partes. **Es una tarea pesada**, ya que le implica mantener estado de ciertos paquetes y sus fragmentos.

Path MTU: cuando dos hosts están separados por más de un hop, las MTUs de los distintos enlaces pueden diferir. Se define como el **mínimo de todos los MTUs involucrados en el camino** de un nodo a otro.



Y las dos propiedades que hay que mencionar (están en la solución oficial del 2010-12):

- **Puede variar con el tiempo** si los caminos cambian (ECMP o recálculo de rutas).
- **Puede ser diferente según el sentido del tráfico** (ruteo asimétrico: un mínimo para la ida y otro para la vuelta).

Parte b) — Comportamiento del router

```
// ---- Router R: paquete Pi de largo L, MTU de salida M ----

if (L <= M): → // CASO 1: cabe entero
transmitir Pi normalmente // (DF es irrelevante)

else if (L > M and Pi.DF == 1): → // CASO 2: prohibido partir
DESCARTA Pi
enviar ICMP (destino = Pi.origen, tipo = 3, código = 4)
// tipo 3 = Destination Unreachable
// código 4 = "Fragmentation Needed and Don't Fragment was Set"
// los bytes 6-7 del ICMP llevan la MTU del enlace bloqueante (RFC 1191)

else if (L > M and Pi.DF == 0): → // CASO 3: se puede partir
FRAGMENTAR Pi
transmitir cada fragmento
// Total Length de cada fragmento = IHL×4 + payload_i
// Fragment Offset_i = offset_acumulado / 8 ← EN UNIDADES DE 8 BYTES
// MF = 1 en todos menos el último (MF = 0)
// Identification = el MISMO en todos
```

La justificación del ICMP tipo 3 código 4 (los puntos clave):

- **Para que el emisor no repita el error.** Sin aviso, el emisor solo ve pérdida y **retransmite el mismo datagrama idéntico** una y otra vez (bucle infinito).
- **Para que el emisor pueda ajustarse.** El mensaje **lleva la MTU exacta** del enlace bloqueante en los bytes 6-7, así que puede reducir su MSS o su caché de PMTU.

Y por qué se descarta con `DF = 1`: el emisor declaró "no me fragmentes", así que el datagrama **tiene un límite de tamaño que hay que respetar**, y el receptor final no sabe tratar datos partidos.

Parte c) — Cálculo de los fragmentos

Datos: datagrama de 1500 bytes, `IHL = 5` (cabecera 20), payload = 1480 bytes, `DF = 0`, pasa por un enlace de MTU **576**.

Paso 1 — payload por fragmento:

```
Máximo por fragmento = 576 − 20 (cabecera) = 556
Redondeo a múltiplo de 8: 556 / 8 = 69,5 → 69 × 8 = 552 (el múltiplo de 8 más grande ≤ 556)
⇐ Payload por fragmento = 552
```

Paso 2 — cantidad de fragmentos:

```
n = ceil(1480 / 552) = ceil(2,681) = 3 fragmentos
```

Los 3 fragmentos:

#	`Fragment Offset` (bytes)	En unidades de 8	Payload	`Total Length`	`MF`
F1	0	0/8 = **0**	552	20 + 552 = **572**	1
F2	552	552/8 = **69**	552	**572**	1
F3	1104	1104/8 = **138**	1480 − 1104 = **376**	20 + 376 = **396**	**0**

Verificación:

```
Suma de payloads: 552 + 552 + 376 = 1480 ✓ (coincide con el original)
Offsets válidos: 0, 69 y 138 — los tres son enteros (552/8=69; 1104/8=138) ✓
MF: 1, 1, 0 ✓ (solo el último con MF=0)
```

Bytes que viajan en el enlace de MTU 576:

```
572 + 572 + 396 = 1540 bytes
Overhead de fragmentación = 1540 − 1500 = 40 bytes (→ 2,7 % del original)
```

Interpretación: por pagar 40 bytes de overhead (dos cabeceras IP repetidas), el datagrama pasó de 1500 a 1540 bytes en el cable. Y el costo real es otro: **si se pierde cualquiera de los 3 fragmentos, se pierde el datagrama entero.**

⚠️ **Y si el mismo datagrama viniera con `DF = 1`?** `L = 1500 > 576` y `DF = 1` → **R1 lo descarta** y manda **ICMP tipo 3 código 4** a A. Cero bytes salen por ese enlace. Y si el Path MTU ya estuviera cacheado en 576, el paquete de A tendría que ser de 576 bytes como máximo.

Parte d) — Algoritmo de descubrimiento del Path MTU

Primitivas: `enviar_msg_ICMP(destino, tipo, código, largo_payload)` *y* `recibir_msg_ICMP(fuente)` *(devuelve* `null` *si pasa un tiempo).*

Solución oficial (multiplicativa):

```
Pmtu = MTU(enlace hacia B) // parte de un valor tentativo
tries = 0
while (true):
enviar_msg_ICMP(destino = B, tipo = 8, largo = Pmtu) // Echo Request
pkt = recibir_msg_ICMP(fuente = B)
if (pkt != null):
if (pkt.code == 0): // es un Echo Reply: llegó entero
return Pmtu
else: // nueva aproximación (agresiva)
Pmtu = Pmtu * 0.85
else: // timeout u otro error de red
Pmtu = Pmtu * 0.85
tries++
if (tries >= 5):
return False
```

Análisis de la solución oficial y mi versión mejorada:

La solución oficial es funcional pero tiene **varios problemas** (que valen puntos si los mencionás):

1. 🔴 `if pkt.code == 0` **sin mirar** `pkt.tipo`**.** El **tipo 3 también tiene código 0** ("Net Unreachable"), como se ve en la propia tabla del examen. Si el router responde tipo 3 código 0, el algoritmo **devuelve un PMTU que nunca funcionó**.
2. 🔴 **No usa la MTU que viene en el ICMP.** El `RFC 1191` la pone en los **bytes 6-7** del tipo 3 código 4. El enunciado da la tabla **justo para que uses ese mensaje**, y la respuesta **ya está dentro del ICMP**.
3. 🟡 `Pmtu * 0.85` **nunca da el PMTU exacto.** Los valores son $1500 \cdot 0{,}85^k$; ninguno es 1492. Con PMTU real 1492 devuelve **1275** (**14,5 % bajo**).
4. 🟡 **No cachea el resultado.** El *almacenamiento en caché* es parte del mecanismo: sin él, cada datagrama grande genera un ICMP.
5. 🟡 **No hay cota inferior.** Puede bajar de 68 (el mínimo legal de IPv4) y seguir mandando sondas imposibles.
6. 🟡 **No distingue "MTU chico" de "ruta rota".** Un tipo 3 código 1 (host unreachable) o 9/10 (prohibido) no se arreglan bajando el tamaño.
7. 🟠 `tries` **solo cuenta en el timeout.** Si el router siempre responde tipo 3 código 4, el `while true` **no termina nunca**.

Mi solución: búsqueda binaria + lectura de la MTU del ICMP + caché:

```
// ======== ESTADO PERSISTENTE EN A ========
pm[B] = { valor, valido, timestamp } // caché de PMTU por destino
MAX_REINTENTOS = 5 ; MTU_MIN_ABSOLUTO = 68

int descubrir_PMTU(A, B):
// ---- Invariantes ----
// lo : tamaño que YA sabemos que pasa entero (cota inferior)
// hi : tamaño que YA sabemos que NO pasa (cota superior)
// Al salir, lo es el mayor tamaño conocido como transmisible.
if (pm[B].valido): lo = pm[B].valor // arrancamos del caché
else: lo = MTU_MIN_ABSOLUTO // 68: seguro por definición
hi = MTU(interfaz de A hacia B) // 1500: no puede pasar más
reintentos = 0

while (hi - lo > 1):
mid = (lo + hi) / 2 // ← búsqueda BINARIA
enviar_msg_ICMP(B, 8, 0, mid) // Echo Request con DF = 1
pkt = recibir_msg_ICMP(B)

if (pkt == null): // timeout
// Degradación elegante: asumimos el PEOR caso (que mid era
// demasiado grande) y seguimos binary-searching.
reintentos++
if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
hi = mid

else if (pkt.tipo == 0 and pkt.codigo == 0):
lo = mid // llegó entero a B: ES válido

else if (pkt.tipo == 3 and pkt.codigo == 4):
// ★ AQUÍ ESTÁ LA MEJORA PRINCIPAL ★
m = leer_uint16(pkt, offset = 6) // Next-Hop MTU (RFC 1191)
if (m >= lo and m < mid): lo = m; hi = m // m ES transmisible
else: hi = mid // mensaje inconsistente

else if (pkt.tipo == 3):
// Otro inalcanzable (net, host, port, prohibido):
// NO es un problema de tamaño, es un problema de ruta.
cachear(B, lo); return ERROR_NO_EXISTE_RUTA

else: // tipo 11, 12, etc.
reintentos++
if (reintentos > MAX_REINTENTOS): cachear(B, lo); return lo
hi = mid

cachear(B, lo)
return lo
```

Lo que arregla mi versión:

Mejora	Por qué
`pkt.tipo == 0 && pkt.codigo == 0`	El tipo 3 tiene un código 0. Chequear solo el código hace que el algoritmo **diga que funcionó cuando no**. Bug real de la oficial
Leo la MTU de los bytes 6-7	El mensaje **ya trae la respuesta exacta**. **1 solo probe** reemplaza a los 8 de la búsqueda binaria, y el resultado es **exacto** en vez de 14,5 % bajo
Búsqueda binaria `(lo+hi)/2`	`O(log n)`: peor caso `log₂(1432)` = **11 probes**, **siempre**. El `× 0,85` nunca converge al valor exacto
Caché por destino	El PMTU es **propiedad de la ruta**. Sin caché, chaque paquete grande vuelve a generar un ICMP
Cota inferior `lo = 68`	Nunca sondea algo que **no puede funcionar**
Distingue los otros tipo 3	Un "host unreachable" no se arregla bajando el MTU
Contar todos los reintentos	La oficial puede no terminar nunca

Simulación con estos datos (PMTU real = 576, MTU local = 1500):

```
lo = 68 ; hi = 1500

mid = 784 → llega entero → lo = 784
mid = 1142 → llega entero → lo = 1142
mid = 1321 → llega entero → lo = 1321
mid = 1410 → llega entero → lo = 1410
mid = 1455 → llega entero → lo = 1455
mid = 1477 → llega entero → lo = 1477
mid = 1488 → llega entero → lo = 1488
mid = 1494 → ICMP 3/4 con MTU=576 → lo = hi = 576 (termina)

Resultado: PMTU = 576 ✓ EXACTO (8 probes)
⇐ En este caso el ICMP con el MTU exacto resolvió todo en 1 paso.
```

> **Tope para los 7 puntos:** el pseudocode completo y ejecutable, la **inicialización del estado**, el **manejo del timeout**, la **lectura del tipo y el código** del ICMP recibido, y **al menos una justificación** de por qué tu estrategia es mejor que "probar y ver qué pasa".

PARTE 4 — CIERRE

📌 Resumen

1. **ICMP existe porque IP no puede reportar sus propios errores** (problema de anidamiento). Los errores van **de nodo a nodo**.
2. **El formato**: tipo (8) + código (8) + checksum (16) + cuerpo. En `ping`, el cuerpo lleva **Identifier** y **Sequence Number**.
3. **Los tipos de memoria**: 0 Echo Reply, 3 Dest Unreachable, 4 Source Quench (**obsoleto**), 5 Redirect, 8 Echo Request, 9 RA, 10 RS, 11 Time Exceeded, 12 Parameter Problem, 13 Timestamp, 17 Address Mask (**obsoleto**).
4. **Los códigos del tipo 3**: 0 net, 1 host, 2 protocol, 3 port, **4 frag needed + DF**, 5 source route failed, 9/10 admin prohibited.
5. **ICMP no es confiable y no genera errores sobre errores** (salvo el 3/4), para no generar tormentas.
6. `ping`: Echo Request (tipo 8) / Echo Reply (tipo 0). Mide RTT y pérdida. `ping -M do -s <tamaño>` **diagnostica el PMTU.**
7. `traceroute` explota el **TTL**: cada router que lo ve llegar a 0 devuelve **ICMP tipo 11, código 0**. En el sabor **UDP** el destino devuelve **tipo 3 código 3**; en el sabor **ICMP**, **tipo 0**.
8. **Los `*` no significan "caído"**: significan "no hay respuesta" (filtrado, pérdida, o el ICMP de vuelta no llegó). Y el **camino de vuelta puede ser distinto del de ida** (ruteo asimétrico), así que los RTTs son del ciclo completo.
9. **Repaso Bloque D**: subnetting (`2^(32-p)-2` usables), LPM, TTL, MTU/MSS (`MTU-40` en IPv4), los 5 mensajes de OSPF, atributos de BGP (LOCAL_PREF ↑, AS_PATH length ↓, MED ↓), los 3 modelos de IPv6.
10. **Errores frecuentes**: confundir MTU con MSS, creer que anycast es un tipo de dirección, creer que IPv6 fragmenta en routers, confundir tipo 11 con 3/3 en traceroute, proponer Source Quench para congestión, creer que los routers reenvían broadcast.

✅ Autoevaluación

1. ¿Por qué los errores de IP no pueden reportarse con IP? ¿Y por qué los routers forwarding no generan un ICMP tipo 11 cuando un ICMP tipo 11 no puede entregarse?

▶ Respuesta

2. ¿Por qué el `Source Quench` (tipo 4) está obsoleto? ¿Qué se usa en su lugar?

▶ Respuesta

3. ¿Cuál es el tipo y código de ICMP que devuelve cada nodo en un `traceroute` sabor UDP, desde el primer router hasta el destino?

▶ Respuesta

4. Un `traceroute` muestra `* * *` en el salto 5 de 8. ¿Qué podés concluir?

▶ Respuesta

5. ¿Por qué los RTTs que muestra un `traceroute` no tienen por qué ser crecientes de un salto al siguiente?

▶ Respuesta

6. En el simulacro, la Teórica 1 tiene `10.0.0.0/24` y `10.0.1.0/24`. ¿Cuál matchea un paquete a `10.0.1.55`? ¿Se ve el longest prefix match?

▶ Respuesta

7. ¿Por qué el router no fragmenta un paquete con `DF = 1` aunque quepa en otro enlace con fragmentación disponible?

▶ Respuesta

8. ¿Cuál es el MSS de una conexión TCP sobre IPv6 en un enlace con MTU 1500, y por qué difiere del de IPv4?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1103` / `sirc1103`** — P3 (6 pts: principio de `traceroute`, tipos de mensaje, nodos que colaboran, limitaciones del `*` y del ruteo asimétrico)
- **Examen `eirc1012` / `sirc1012`** — Problema 2 (30 pts, con la **tabla de tipos y códigos de ICMP** que es material de estudio); P5 (IPv6)
- **Examen `sirc0612`** — P3 (fragmentación y reensamblaje)
- **Examen `sirc0703`** — P1 (5 mensajes de OSPF); P3 (RARP)
- **Examen `sirc0807`** — P2 (protocolos de enrutamiento)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.4.1 ("ICMP — El protocolo de control de la capa de Internet") y cap. 5 en general
- **RFC 792** — ICMP (la tabla de tipos y códigos que hay que saber de memoria)
- **RFC 1122** — Requirements for Internet Hosts (comportamiento de los hosts y de los routers; reglas de ICMP)
- **RFC 4443** — ICMPv6 (tipos 1-4 y 128-137)
- **RFC 1191** — Path MTU Discovery (el MTU del enlace bloqueante en los bytes 6-7 del ICMP)
- **Clase siguiente:** 38 — Capa de enlace: servicios y framing